

# Aussie EcoLens

Multi-Cloud Serverless  
Wildlife Observation Platform

AWS

GCP Model

ML Tags

## Serverless workflow



Upload  
AWS Lambda



Detect  
Cloud Run



Notify  
SNS email

Searchable wildlife records

### Submission Note

This PDF contains the team report narrative, the official-icon multi-cloud architecture diagram, rubric coverage evidence, a testing guide, CI evidence, and a contribution table template. Replace the highlighted team-member placeholders with real names, student IDs, and contribution percentages before Moodle submission.

## FIT5225 2026 S1 Assignment 2

Final Team Report and Verification Evidence

Private source repository: shared with teaching team separately

# 1. Overview

Aussie EcoLens is a multi-cloud online storage and retrieval system for wildlife media. Authenticated users can upload camera-trap images and videos, receive automatic species tags from the supplied course ML model, query media by tag counts or species, map thumbnail URLs back to full images, query by an uploaded file without storing it, bulk edit tags, delete media, and receive watched-tag notifications. The final deployment deliberately splits responsibilities across two clouds: AWS provides the secure application plane, while Google Cloud runs the heavy-weight model container and stores mirrored metadata. This design keeps the API serverless and protected by Cognito while avoiding a large model package inside Lambda.

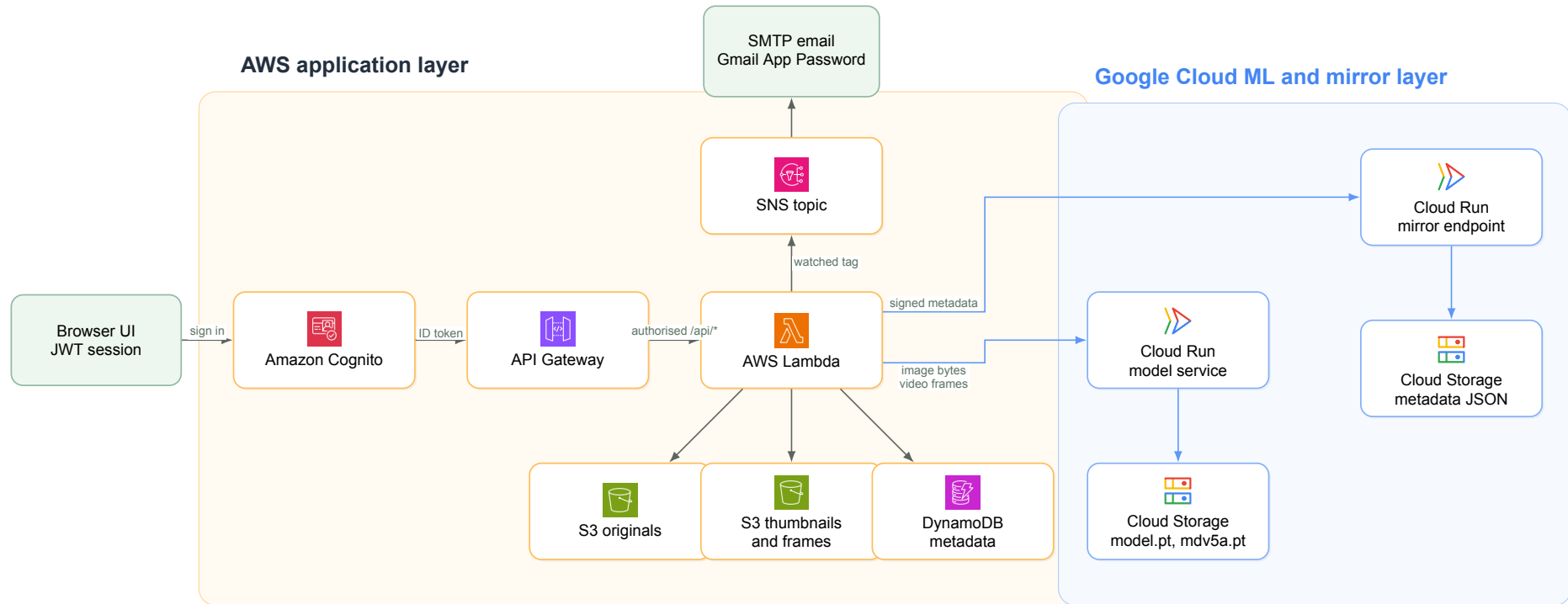
## Live Deployment Snapshot

|                     |                                                                                             |
|---------------------|---------------------------------------------------------------------------------------------|
| <b>AWS region</b>   | ap-southeast-2: Cognito, API Gateway, Lambda, S3, DynamoDB, SNS                             |
| <b>GCP region</b>   | australia-southeast1: Cloud Run model service and metadata mirror                           |
| <b>ML runtime</b>   | Python 3.12, PyTorch, torchvision, MegaDetector, onnx2torch, supplied <code>model.pt</code> |
| <b>Notification</b> | In-app records, SNS publish path, and verified SMTP email through Gmail App Password        |

## Verified Evidence

Cloud smoke testing confirmed unauthenticated API rejection, image upload with `casuarius_casuarius: 1`, checksum duplicate detection, thumbnail-to-full-image lookup, query-by-file without storage, manual tag edit, deletion, video frame extraction at one frame per second, GCP metadata mirroring, and watched-tag notifications through `in_app`, `sns`, and `smtp`. Local CI tests verify the same API contract without requiring cloud credentials.

## 2. Multi-Cloud Architecture



The diagram uses official AWS Architecture Icons and official Google Cloud product icons. AWS contains the security and application plane: Cognito protects all workflows, API Gateway verifies JWTs, Lambda processes uploads and queries, S3 stores originals and derivatives, DynamoDB stores searchable metadata, and SNS participates in watched-tag notifications. Google Cloud provides the secondary-cloud plane: Cloud Run loads the supplied model files from Cloud Storage, predicts species for images and video frames, and another Cloud Run endpoint mirrors metadata into Cloud Storage.

### 3. Design and Implementation Choices

The system is serverless because the workload is naturally event-driven: uploads, query requests, tag edits, deletes, and notification watches arrive intermittently and should not require a managed virtual machine. AWS Lambda calculates a SHA-256 checksum before creating meta-data, so repeated uploads return the existing record rather than wasting storage. Image thumbnails are compressed while preserving aspect ratio. Video uploads are sampled with ffmpeg at one frame per second, and each extracted frame is sent to the same course model pipeline as image uploads. Model handling is intentionally configurable: Cloud Run reads classifier and detector blob names from environment variables, so a future `model.pt` or `mdv5a.pt` can be swapped by changing Cloud Storage object paths rather than editing Lambda source code.

#### No Material Shortcuts

The final cloud mode uses the supplied ML model, not filename-only tagging. Filename tagging remains only as a deterministic local fallback for CI and offline demonstration. The cloud smoke test exercised real GCP model inference, AWS storage, DynamoDB metadata, Cognito-protected APIs, GCP mirroring, video frame extraction, and verified SMTP notification delivery.

## 4. Rubric Coverage Matrix

| Rubric item                              | Implemented evidence                                                                                                                                                                                                                          | Verification path                                                                                                                             |
|------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Cognito auth and sign-out</b>         | AWS Cognito Hosted UI, email attributes, first and last name fields, JWT tokens, sign-out button.                                                                                                                                             | Sign up or sign in through the Cognito domain; UI shows the authenticated user and protected workflows.                                       |
| <b>Access control and IAM</b>            | API Gateway JWT authoriser protects <code>/api/*</code> . Lambda role is scoped to S3 buckets, DynamoDB table, SNS topic, Rekognition where enabled, and CloudWatch logs.                                                                     | Unauthenticated API smoke test returns 401; SAM template defines managed policies and bucket/table/topic permissions.                         |
| <b>Upload and deduplication</b>          | Authenticated upload stores originals in S3. SHA-256 checksum is the media key and duplicate uploads return existing metadata.                                                                                                                | Upload the same file twice; UI reports duplicate and does not create a second metadata record.                                                |
| <b>Image thumbnails and video frames</b> | Lambda creates compressed JPEG thumbnails for images and extracts video frames with <code>fps=1</code> ; frames are stored in the thumbnail bucket.                                                                                           | Upload an image and video; verify thumbnail URL, frame URLs, and roughly one frame per second.                                                |
| <b>ML tagging and DB insertion</b>       | AWS Lambda calls GCP Cloud Run; Cloud Run loads <code>model.pt</code> and <code>mdv5a.pt</code> from GCP Storage and returns species counts. DynamoDB stores media type, tags, original key, thumbnail key, frame keys, owner, and timestamp. | Cloud test returned <code>casuarius_casuarius: 1</code> ; video frames are sent to the same model endpoint.                                   |
| <b>Queries</b>                           | APIs implement tag-count AND search, species search, thumbnail-to-full URL lookup, and query-by-file without storage.                                                                                                                         | UI query panels and local CI tests exercise each endpoint.                                                                                    |
| <b>Bulk tag edit and delete</b>          | POST tag edit accepts URL list, tags, and operation 1 or 0. Delete removes originals, thumbnails, video frames, and DynamoDB items.                                                                                                           | UI manage panel plus local tests for bulk remove and deletion.                                                                                |
| <b>Notifications</b>                     | Watch records trigger in-app notes and publish via SNS and SMTP email.                                                                                                                                                                        | Cloud smoke test returned channels <code>in_app</code> , <code>sns</code> , and <code>smtp</code> ; recipient confirmed receiving SMTP email. |
| <b>UI completeness</b>                   | Single-page UI supports auth, image/video upload mode switch, batch upload, queries, thumbnail preview, URL copy/show controls, tag editing, delete, and notifications.                                                                       | Demonstrate every workflow from <code>README_TESTING.md</code> .                                                                              |
| <b>Reports and repository</b>            | This PDF includes official icons, contribution table, testing guide, and source repository evidence. Detailed product manual and CI workflow are included in the repository.                                                                  | Private repository access is shared with the teaching team separately.                                                                        |

## 5. Testing, CI, and Reproducibility

### Automated CI

The repository includes `.github/workflows/ci.yml`. On push or pull request, GitHub Actions installs Python 3.12 and Pillow, compiles Python sources, checks deployment shell scripts with `bash -n`, and runs `python -m unittest discover -s tests -v`. The tests cover protected-route rejection, local sign-up/sign-in, upload, checksum deduplication, tag-count AND query, species query, thumbnail lookup, query-by-file, watched-tag notifications, bulk tag removal, deletion, and video metadata upload.

| Test type                 | Purpose                                                             | Command or evidence                                                                         |
|---------------------------|---------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| <b>Local CI workflow</b>  | Reproducible verification without cloud credentials.                | Python unittest discovery over <code>tests/</code> .                                        |
| <b>Cloud smoke test</b>   | Confirms deployed AWS/GCP services and real ML model integration.   | Verified image upload, duplicate handling, query-by-file, deletion, GCP mirror, SMTP email. |
| <b>Manual demo script</b> | Gives teaching staff a predictable demonstration order.             | README testing guide and demo walkthrough.                                                  |
| <b>Deployment scripts</b> | Ensure cloud infrastructure can be recreated by an authorised user. | CI syntax-checks the AWS, GCP, and model-service deploy scripts.                            |

## 6. User Guide for Testing

| Workflow                | Steps                                                                       | Expected output                                                                            |
|-------------------------|-----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
| <b>Start UI</b>         | Run <code>python3 -m src.aussie_ecolens.server; 127.0.0.1:8080.</code> open | Page loads and redirects to Cognito in cloud mode.                                         |
| <b>Sign in</b>          | Register or sign in through Cognito Hosted UI.                              | User name and Sign out button appear; protected tools are visible.                         |
| <b>Image upload</b>     | Select Image mode, choose one or more images, click upload.                 | Result cards show thumbnails, tag chips, duplicate status, and copyable Thumbnail URLs.    |
| <b>Video upload</b>     | Select Video mode and upload a short video.                                 | Original video URL, extracted frame previews, frame URLs, and aggregated tags appear.      |
| <b>Tag query</b>        | Submit JSON such as <code>{"casuarius_casuarius": 1}</code> .               | Results contain media whose counts meet all requested tags.                                |
| <b>Species query</b>    | Search for a species name such as <code>casuarius_casuarius</code> .        | All matching images and videos are returned.                                               |
| <b>Thumbnail lookup</b> | Paste a Thumbnail URL and click Get full image.                             | UI returns an openable, copyable full original image URL.                                  |
| <b>Query by file</b>    | Upload a query-only image.                                                  | Detected tags are used to search existing media; the query file is not permanently stored. |
| <b>Tag edit/delete</b>  | Paste one or more media URLs, add/remove tags, then delete.                 | Query results update, and delete removes metadata and stored derivatives.                  |
| <b>Notifications</b>    | Watch a species tag, upload matching media, refresh Notifications.          | In-app note appears and verified SMTP email is sent.                                       |

## 7. Packaging and Remaining Submission Tasks

### Zip Package Reproducibility

A recipient can unzip the repository, install Python 3.12 plus Pillow, run `make test`, and start the local web demo without AWS or GCP credentials. Cloud redeployment requires AWS and GCP login plus the deployment variables documented in the LaTeX user manual. Secrets are intentionally excluded from the repository.

### Only Non-Technical Placeholders Remain

The implementation and technical documentation are complete. Before Moodle submission, replace the contribution table placeholders with real names, student IDs, percentages, and contribution descriptions. Each student must also submit their own individual report.

## 8. Team Contributions

| Name and ID           | Contribution % | Project elements contributed                                                                                                          |
|-----------------------|----------------|---------------------------------------------------------------------------------------------------------------------------------------|
| TBD: Name, Student ID | TBD %          | Replace with each member's real contribution summary, maximum 100 words per member.                                                   |
| TBD: Name, Student ID | TBD %          | Replace with authentication, cloud deployment, ML integration, UI, testing, documentation, or reporting contributions as appropriate. |
| TBD: Name, Student ID | TBD %          | Replace before final Moodle submission.                                                                                               |

## 9. Repository, References, and AI Statement

Private source code repository access is shared with the teaching team separately. Keep the repository private to avoid plagiarism exposure.

**Icon sources.** AWS icons were downloaded from the official AWS Architecture Icons page: <https://aws.amazon.com/architecture/icons/>. Google Cloud icons were downloaded from the official Google Cloud product icons library: <https://cloud.google.com/icons>.

**Generative AI acknowledgement.** Generative AI assistance was used selectively for implementation support, debugging, documentation drafting, and report layout. The team remains responsible for verifying the final code, cloud configuration, tests, and submitted text.